

Distributed Ledger System for Managing Delegated Authority and Revocations

This document describes an original project authored by Tomás Lázaro Danoni Mazurek on August 10, 2025. Any reproduction or use must credit the author.

Abstract

A blockchain-based authorization management system that lets users appoint cryptographic agents to act on their behalf in domains such as banking, voting, defense, and civic administration. Agent permissions are established via digitally signed transactions and stored on a tamper-evident distributed ledger. A trusted revocation authority (e.g., a bank, court, or government agency) holds a special key allowing it to revoke an agent's authorization but not assign new agents or perform other actions for the user. This one-way control limits damage from compromised keys or rogue agents by enabling rapid removal of their powers while preserving user autonomy. The design combines decentralized trust with a limited centralized failsafe, using cryptographic protocols to ensure only authorized parties can update the ledger. Example applications span multiple critical domains, enhancing both security and recoverability of delegated authorizations.

Technical Field

This project relates to the field of **distributed ledger technology** and digital security, particularly to mechanisms for **identity and permission management** on blockchains. It addresses technical problems in **cryptographic key management**, **delegation of authority**, and **revocation of permissions** in decentralized systems. Example domains of application include but are not limited to: digital banking and asset custody, electronic voting systems, military command-and-control networks, and governmental or civic identity systems that use blockchain for record-keeping.

Background

Public-key cryptography allows individuals to authenticate and authorize actions in digital systems (e.g. sign transactions, votes, or commands). Blockchains provide a tamper-resistant ledger for recording such actions and the state of system permissions. In many scenarios, a user (principal) may wish or need to delegate their authority to an agent (which could be another human, an organization, or a software process) to act on the user's behalf. For instance:

- In banking or asset management, an elderly or busy account holder might authorize an agent (like a financial advisor or automated service) to manage their cryptocurrency wallet or digital assets.
- In voting, a citizen might delegate their vote to a proxy (a form of liquid democracy) or an aid if the citizen is incapacitated.
- In military/defense, a commander could delegate certain decision-making or system access to an AI agent or a junior officer via a secure ledger-based system.
- In government services, a citizen might appoint an agent (e.g. legal guardian or representative) to interact with e-government portals on their behalf.

Delegation is convenient but introduces security risks. If the agent's key is compromised or the agent becomes malicious, the principal could suffer harm (financial theft, misuse of voting power, unauthorized military orders,

etc.). Ideally, the principal would simply revoke the agent's authority upon detecting a compromise. Indeed, blockchain-based delegation systems exist where the delegator can broadcast a revocation transaction to invalidate a previously authorized agent key [2]. However, a critical failure mode remains unaddressed by user-only revocation: *what if the user themselves loses access to their private key or is otherwise unable to issue a revoke?*

For example, if a user's own key is lost, stolen, or held hostage, the user **cannot sign a revocation transaction** to remove a rogue agent. The agent (or an attacker wielding the agent's credentials) could continue acting with the user's authority indefinitely. In high-stakes domains, this is unacceptable. A purely decentralized system has no fail-safe in this scenario - **assets or powers could be irreversibly misused**.

Centralized institutions (like banks or courts) have long been entrusted to intervene in extraordinary cases (freezing accounts, voiding fraudulent transactions, revoking power-of-attorney, etc.). But naive introduction of centralized control into blockchain systems can undermine their security and user autonomy. If a bank or government could arbitrarily assign themselves or others as agents on user accounts, that would violate the user's sovereignty and invite abuse. Thus, there is a delicate balance: **introduce a trusted third-party with limited emergency powers**, strictly constrained to revocation actions only.

To date, there has been **no integrated solution** that leverages blockchain's transparency and auditability while incorporating an *oversight authority restricted to revocations*. Existing permissioned ledger solutions allow admins to revoke user credentials [8], but those operate in closed systems and often make the admin all-powerful (able to assign or change roles at will). In contrast, the present project restricts the authority's role to **only removing permissions**, never granting them. This preserves the user's ultimate control over whom to trust as an agent, while still providing a safety net if things go wrong.

In summary, the project arises from the need to **enhance the security of delegated authority on blockchains** by enabling a *trusted institution to serve as a revocation guardian*. This approach aims to mitigate the impact of key compromise or user incapacitation in a multi-domain context without reverting to a fully centralized model. The technical challenge is designing a ledger and smart contract architecture that enforces these asymmetric rights (user can assign agents; authority can only revoke) in a verifiable way.

Summary of the Project

The project provides a **blockchain-based system and method** for managing delegated authorizations (agents) with a built-in trusted revocation mechanism. At a high level, each user on the system can appoint one or more **agents** by adding an entry on a blockchain ledger (for example, in a smart contract or a special transaction type) that links the agent's cryptographic public key to the user's identity along with the scope of permissions. This forms an **on-chain authorization record** indicating that "Agent X is authorized to act for User Y" under certain conditions.

Crucially, the system designates a **trusted institutional authority** (different for different contexts - e.g. a bank for financial transactions, an election commission for voting, a military command authority for defense) that holds a special **revocation private key**. The corresponding public key is recognized by the ledger or smart contract as having the ability to **cancel (revoke) agent authorizations**, but *not* to create or reassign them. In other words, the authority's key can publish a transaction or call a contract function that **strikes out an agent authorization entry**, marking it invalid from that point forward. The authority cannot appoint a new agent on behalf of the user; only the user's own key is allowed to add or modify agent assignments (except revocation).

Additional features and aspects of the project include:

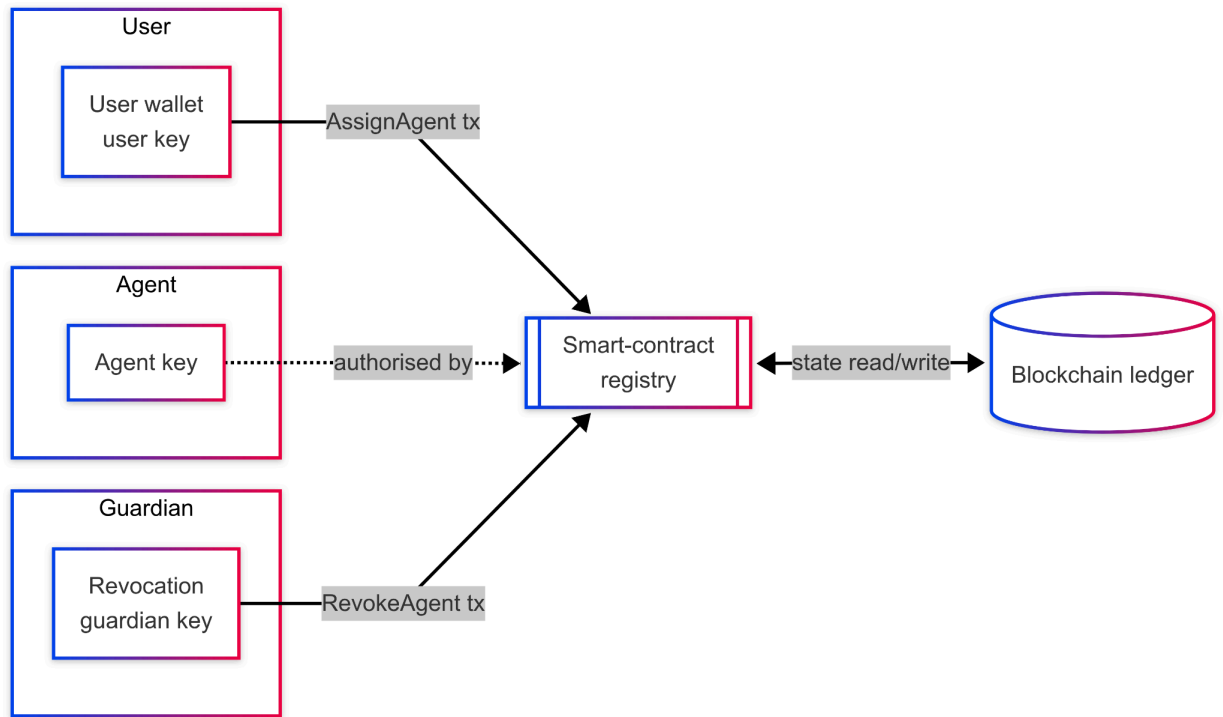
- **Ledger-Stored State:** The current state of all agent authorizations is maintained on the blockchain (for example, as a mapping in a smart contract: `principal → agent → permissions/ status`). This state is updated by authorized transactions. Thus, anyone can query the blockchain to determine who is the authorized agent of a given user at any time, and whether a given agent's authority has been revoked. The ledger acts as the **source of truth** for agent roles.
- **Dual-Key Security Model:** Each user account is associated with two types of keys: (1) the **user's key** (master key) which can authorize agents (and perform normal transactions), and (2) the **revocation authority's key** (a well-known key held by the trusted institution) which can revoke the user's agent. The smart contract logic or transaction validation rules ensure that only these respective keys can perform their specific operations. For example, an "AssignAgent" transaction must be signed by the user's key, and a "RevokeAgent" transaction must be signed by the authority's key (for the relevant domain). Any transaction not meeting these criteria is rejected by the network consensus.
- **One-Way Power of Authority:** The institutional authority is *per user or per domain*. In some implementations, each user could register an approved "revocation guardian" (e.g. a court or agency they trust) when they onboard, or the system might have a fixed authority per domain (e.g. the national election board for all voting delegations). In all cases, that authority is **cryptographically prevented** from exercising the user's privileges (they cannot impersonate the user or their agent; they cannot transfer assets or cast votes) - they can only issue a **revoke command** that targets an existing agent link. This one-way control is enforced at the smart contract level and can be audited on-chain.
- **Revocation Process:** If a user detects compromise of their agent (or loses their own key), they alert the revocation authority through an out-of-band mechanism (e.g. contacting the bank or authority with proper identification, or perhaps a pre-arranged secret). The authority then uses its private key to create a **Revocation Transaction** on the blockchain. This transaction specifies the user ID and/or agent ID to be revoked (it may reference the original authorization entry by ID or hash). When this transaction is confirmed on-chain, the state is updated such that the agent is no longer authorized for that user. Any subsequent actions by that agent on behalf of that user are **rejected by the network or smart contract**, since the ledger now shows no valid authorization. Essentially, the agent's power is nullified almost immediately in an auditable, irrevocable manner (the revocation itself is an immutable ledger event).
- **Notification and Recovery:** Optionally, the system can notify the user and possibly the agent of the revocation (e.g. via an event emission or messaging system tied to the smart contract). After revocation, the user may recover by either (a) using a newly secured key to designate a new agent, or (b) if the user's own key was lost, going through a key recovery process (which could be integrated with the authority verifying the user's identity and updating the ledger to attach a new public key for the user - this aspect can be an extension where the authority's involvement may be used to facilitate user key reset, though the core project focuses on agent removal).
- **Multi-Domain Support:** The design is flexible to accommodate various domains:
 - *Financial Domain:* A bank or financial regulator acts as the revocation authority for agented bank accounts or wallets. For instance, if Alice authorizes Bob to spend from her account via a blockchain tokenized bank ledger, the bank can revoke Bob's access if fraud is suspected, without seizing funds or appointing anyone new.
 - *Voting Domain:* The election authority can revoke a proxy vote delegation if required (e.g. court order

due to abuse of proxy), ensuring the voter's vote isn't misused. The voter could then vote directly or assign a new proxy if time permits.

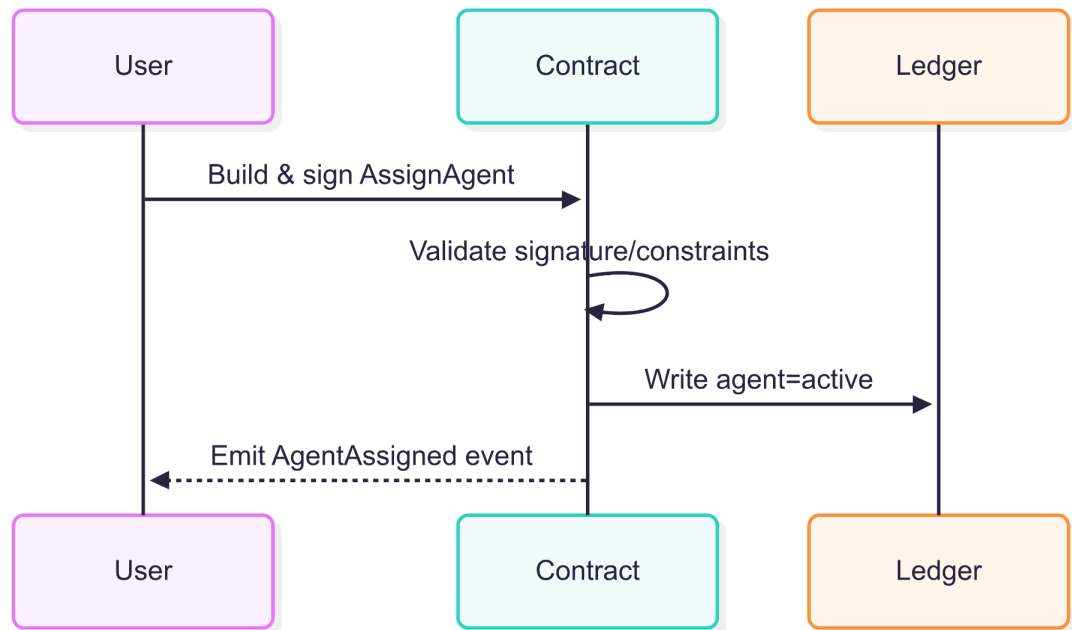
- *Defense Domain:* A military blockchain might allow a commander's AI assistant to issue certain commands. A designated high-level authority (or automated safeguard system) can revoke the AI's command privileges if it malfunctions or is compromised, thereby preventing unauthorized military actions.
- *Civic Administration:* In digital identity systems, a government might allow citizens to empower a trusted family member or lawyer to act on their behalf for accessing services. If that agent exceeds authority or is compromised, a government agency (with proper legal process) can revoke the delegation on the ledger, protecting the citizen. Notably, the agency cannot assign a different representative - the citizen must do that once they regain control.
- **Integrity and Auditability:** Every authorization and revocation is a signed transaction recorded on the ledger. This creates an audit trail: one can prove exactly when and by whom an agent was appointed or revoked. In disputes, the ledger provides evidence (e.g., if an agent claims they still had authority at a certain time, the ledger's history will confirm if a revocation had already occurred). The design leverages blockchain's immutable history to enhance **accountability** for both users and authorities. The authority's actions are public and limited - any attempt by the authority to do anything other than revoke (e.g. assign a new agent illegitimately) would appear as an unauthorized transaction and be rejected by the network or flagged by auditors.
- **Security Considerations:** The authority key must be secured, since it has the power to unilaterally revoke agents. In practice, this key could be a multi-signature key controlled by a committee or a hardware security module in a high-security environment to prevent misuse. The system may also implement rate-limits or require justifications (stored on-chain or off-chain) for each revocation to ensure the authority is exercising power appropriately. On the user side, the mechanism encourages users to use agents for daily operations while keeping their master key offline; knowing that if their agent misbehaves or is compromised, the authority can promptly cut off the agent. This reduces the need for users to risk using their master key frequently, thereby improving overall security.

In sum, the project marries the **self-sovereignty of blockchain users (who choose their agents)** with a **safety net provided by a constrained centralized actor (who can only revoke when needed)**. This hybrid approach is novel and advantageous in scenarios where purely user-driven recovery is insufficient. The features described above and in the detailed description below are intended to be illustrative of the various aspects of the system. These and other benefits of the project will become clear from the following detailed description and the accompanying claims.

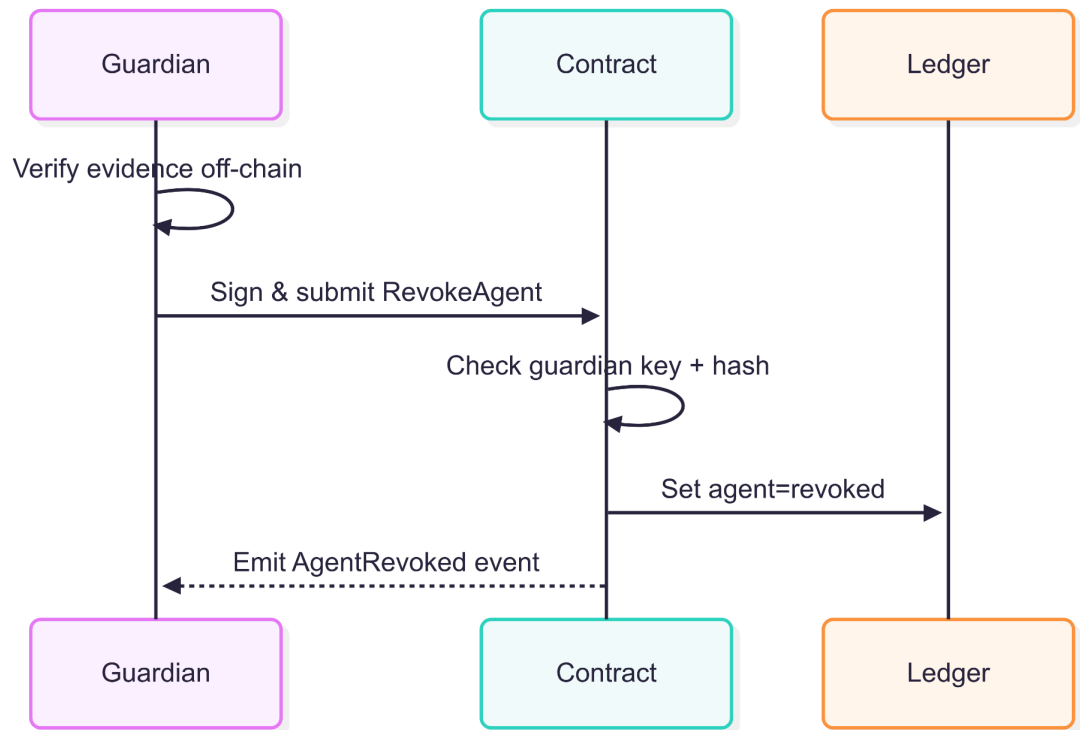
Brief Description of the Drawings



- **FIG. 1** illustrates the overall system architecture. It shows a blockchain network (ledger) with multiple nodes maintaining the state of agent authorizations. A user (principal) is depicted with their **user private key**, which they use to authorize an **Agent Assignment Transaction** that adds or updates an agent's entry in the ledger. An **Agent** (with its own key pair) is linked to the user in the ledger state, meaning it can perform actions on behalf of the user under specified permissions. A **Revocation Authority** (e.g., a bank or other institution) is shown with its separate **revocation key**. FIG. 1 highlights that the authority's key has a unidirectional link to the user-agent link - indicating it can break that link (revoke) but not create a new one. Components like smart contract modules (for validation of assignments and revocations) are also shown.



- FIG. 2** is a flow diagram of the **Agent Assignment process**. In step 2-1, the user decides to delegate to an agent and creates an assignment transaction containing the agent's identifier (public key) and the scope of authority (for example, "can spend up to 5 BTC" or "may vote on proposal X"). In step 2-2, the transaction is signed by the user's private key. In step 2-3, it is submitted to the blockchain network. In step 2-4, nodes validate that the signature matches the user's address and that the user hasn't exceeded any agent limits (e.g., perhaps only one agent allowed at a time, depending on policy). In step 2-5, the ledger state is updated to record the new agent and its permissions as active.



- FIG. 3** is a flow diagram of the **Revocation process**. In step 3-1, a trigger event occurs: this could be the user reporting a lost key or the authority detecting suspicious activity from an agent. In step 3-2, the revocation authority composes a revocation transaction, specifying the user's identity and/or the agent's identity to be revoked (and referencing the original delegation record). In step 3-3, the authority signs this transaction with its **revocation private key**. Step 3-4: The transaction is submitted to the blockchain network. Step 3-5: The network validates that the signature corresponds to the designated revocation authority for that user/domain (each user's record may include the public key of the allowed revocation authority). If valid, the transaction is executed: the smart contract/ledger state marks the agent as revoked (or removes the agent entry entirely) in step 3-6. FIG. 3 also shows that any pending actions by the agent will fail after this point because the agent no longer appears as authorized in the state. In step 3-7, an optional notification is sent to relevant parties (the user and agent) about the revocation.

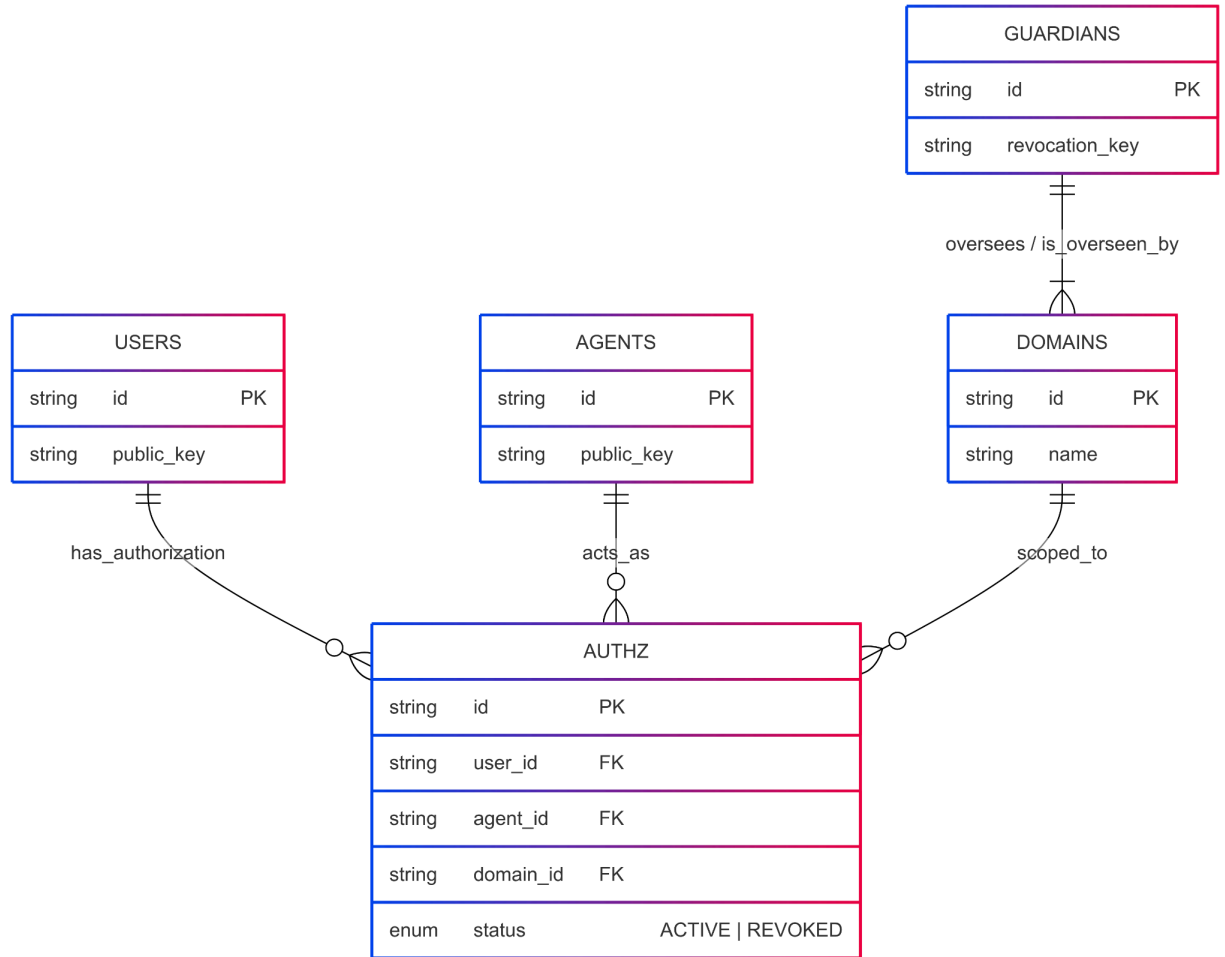


FIG. 4 provides a schematic example of how multi-domain agent management can be implemented. It shows a table or mapping on the ledger with columns for “User”, “Agent”, “Domain/Scope”, “Status”. For instance, User=Alice might have Agent=Bob for Domain=Banking (Status=ACTIVE) and Agent=Carol for Domain=Voting (Status=ACTIVE). If Carol’s authorization was revoked by the Election Commission, the table shows Status=REVOKED for that entry (or the entry is struck through). The figure illustrates that different domains may have different revocation authorities (e.g., a Bank for the banking domain row, an Election Commission for the voting domain row, each with their own key recognized by the contract for revocations in that row). The ledger smart contract enforces that, say, the bank’s key can only affect the banking row, not the voting row, etc.

These drawings and descriptions are provided to facilitate understanding of the project’s structure and processes. They are exemplary and not limiting; those skilled in the art will recognize variations and specific implementation details that can be adapted for particular blockchain platforms (Ethereum smart contracts, Hyperledger Fabric chaincode, etc.) without departing from the scope of the project.

Detailed Description of Embodiments

System Architecture

In one embodiment, the system is implemented as a **smart contract** on an existing blockchain platform that supports programmability (such as Ethereum, Hyperledger Fabric, EOS, or others). In another embodiment, the functionality is built into a custom blockchain protocol (with native transaction types for delegation and revocation). The following description will use the terminology of smart contracts and transactions for concreteness.

Ledger Data Structures: The blockchain maintains a **mapping or registry** of agent authorizations. For example, a smart contract `AgentRegistry` could hold a data structure like:

```
mapping(UserAddress  $\Rightarrow$  AgentInfo) public agentOf;
```

where `AgentInfo` may include fields: `agentAddress`, `domain` (or scope of permissions), and `active` (boolean or status enum). Alternatively, if multiple concurrent agents per user are allowed, it could be `mapping(User  $\Rightarrow$  mapping(domain  $\Rightarrow$  AgentInfo))` or a list of (agent, domain) pairs. For simplicity, assume at most one agent per user per domain in this description.

Each **user** has an on-chain identifier (which could be a blockchain address derived from their public key, or a DID, etc.). Each **agent** likewise has a public-key identifier. The **revocation authority** for a given user (or for a given domain globally) is also identified by a public key, which is either hard-coded in the contract for that domain or stored in a separate mapping (e.g., `revoker[domain] = revocationAuthorityAddress`). The contract is initialized such that for each domain it knows the one and only key that can perform revocations in that domain.

User Enrollment: A user who wants the benefits of this system may register with the relevant authority off chain (for example, go through KYC at a bank or register with the election authority). During this enrollment, the user's blockchain address is linked to the authority in the contract's records. For instance, the bank might call a contract method `registerUser(userAddress)` which stores internally that "for userAddress, the permitted revocation authority is BankAddress". This prevents malicious authorities from revoking random people, they must be explicitly associated (likely with user consent or system policy).

Delegation (Agent Assignment) Procedure

When a user decides to delegate to an agent:

1. **Preparing the Transaction:** The user uses a secure application (wallet software or a specialized interface) to create an **Agent Assignment Transaction**. This contains:
2. The user's identifier (implicitly provided by the signature or explicitly stated).
3. The agent's identifier (public key or address).
4. The domain or scope of the delegation (e.g., "financial", "voting:Election2025", or a specific smart contract address the agent can interact with on behalf of user).
5. Optionally, constraints such as an expiration time for the delegation or limits (for instance, "this agent can transfer up to 10 tokens per day" - such constraints can be enforced by the contract if specified).

The transaction may call a function `assignAgent(agentAddress, domain, [permissions])` on the `AgentRegistry` contract.

1. **User Signature:** The user signs this transaction with their private key. The blockchain network will verify that the signature corresponds to the user's address, proving that the genuine user approved the delegation. The transaction includes the user's public key (or the address which is derived from it) in its `from` field, so the contract knows which user's record to update.
2. **Contract Validation:** When the `assignAgent` function executes (upon being mined into a block), the smart contract code first checks:
 3. That the caller is indeed the user (i.e., `msg.sender == userAddress` or using `ecrecover` on the signature to get the public key).
 4. That the user's account is in a state that allows delegation (for example, the system might disallow assigning a new agent if one is already active unless the user explicitly replaces them, depending on policy).
 5. If multiple domains or roles are allowed, the contract might ensure the `domain` field is valid and maybe that the user is eligible for that (for instance, only a registered voter user can delegate in the voting domain).
 6. Any provided constraints (like an expiration) are recorded.

If any check fails (e.g., signature invalid, user not allowed to delegate, or if the user is flagged as requiring additional authorization by the authority), the function will revert (abort), and no state change occurs.

1. **State Update:** Assuming validation passes, the contract updates the mapping for that user. For example:

```
agentOf[userAddress] = AgentInfo(agentAddress, domain, true);
```

If there was a previous agent recorded, it may either overwrite it (implicitly revoking the previous agent - the contract could emit an event indicating that the previous agent is replaced) or the contract could require the user to explicitly revoke first. In a simple design, a new `assignAgent` just replaces the old agent. The contract marks the status as `active`.

2. **Event Emission:** The contract emits a **DelegationEvent** (e.g., `event AgentAssigned(user, agent, domain)`), which is logged on-chain. This helps external monitoring services or UIs to pick up the change and inform relevant parties in real-time.
3. **Agent's Usage:** Now that the agent is set, the agent can start performing actions on behalf of the user. How this is enforced depends on context:
 4. If the blockchain itself is aware of the delegation (as in some protocols that have a `delegate` field in transactions [1]), the agent could directly sign transactions indicating they act for the user. The network, upon seeing that, will consult the `AgentRegistry` to confirm that the agent is indeed authorized and allowed to do that action. For example, the agent might submit a transaction `Tx(from=User, delegate=Agent, action=Transfer X coins)`. The validation logic would check that `AgentRegistry.agentOf[User]` matches the Agent and is active. If yes, the transaction is accepted and processed as if User performed it (perhaps with some tag that it was via delegation).
 5. In a smart-contract-driven approach, the agent might interact with contracts that require proof of authorization. For instance, a secure vault contract could enforce that any withdrawal by an agent must call `AgentRegistry.isAuthorized(user, agent)` which returns true only if that agent is currently active for that user. This way, even if an agent's key is known, they cannot misuse it unless the

registry affirms their status.

The specifics will differ per use case (for voting, an agent might submit a vote transaction on behalf of user; for banking, an agent might transfer funds from user's account contract; etc.), but the common element is that the **ledger's knowledge of the delegation** gates the agent's actions.

Revocation Procedure

If at any point the user wishes to revoke their agent (while still in control of their key), they could simply issue another assignment to "null" or call a dedicated `revokeMyAgent()` function on the contract. However, the focus here is the scenario where the **user cannot do it themselves**, so the **Revocation Authority** steps in.

Steps for third-party revocation:

1. **Triggering Conditions:** The revocation may be triggered by:
2. A direct request from the user to the authority (e.g., Alice calls her bank hotline and authenticates herself via off-chain means, saying "revoke my agent now!" because she lost her device).
3. Detection of anomalous behavior by the authority's monitoring systems (for example, an AI system flags that Alice's agent is draining funds in an unusual pattern or an election commission system detects an agent casting votes in a suspicious manner).
4. A legal order or emergency condition (court order to revoke an agent's power of attorney, or military high command deciding to revoke an automated system's authority due to a malfunction).
5. **Authority Constructs Revocation Transaction:** The authorized institution uses its **revocation private key** to create a transaction calling a contract function, e.g. `revokeAgent(userAddress)` (the contract can infer the domain or use separate functions per domain if needed). The transaction is **signed by the authority's key**, not the user's. It might include details like a reason code or reference to an authorization ID, but at minimum it identifies the user (and possibly which agent, if the system allowed multiple agents - but if only one agent per user per domain is allowed, the contract already knows which agent to revoke).
6. **Network Validation:** When this revocation transaction is received by the blockchain nodes, the validation logic (within the smart contract or built-in) will check:
7. The signature corresponds to a known revocation authority's public key. Typically, the contract would check `require(msg.sender == revocationAuthorityFor[userAddress])` or if using signature verification, that the signer matches the authority. If the authority for that domain is unique and global, `msg.sender` could be directly compared to that address.
8. Optionally, that the user in question currently has an active agent. (If not, the revocation might be a no-op or return an error "no agent to revoke". But an authority might still issue a revocation just to be sure.)
9. Ensure that the authority is not trying to do anything beyond revocation. If the function is specifically `revokeAgent`, that's inherently limited. The contract would not provide any function for the authority to add agents, so there's simply no code path for misuse in that way.
10. **State Update:** Upon successful validation, the contract updates the state for that user's agent:
11. It could set `agentOf[user].active = false` or remove the entry (`agentOf[user]` is set to a null or default value indicating no agent).

12. It records internally the timestamp of revocation and the revoking party (which might be inferred as the known authority). This could be useful for audit or if later the user is allowed to challenge the revocation (though likely the user requested it or it was per policy).
13. If a reason code or reference was supplied, it might be stored in a log for transparency (e.g., an event).
14. **Event Emission:** The contract emits a **RevocationEvent**, such as `AgentRevoked(userAddress, agentAddress, domain, revocationAuthority)`. This lets client applications know that the user no longer has that agent. For instance, wallet software watching the blockchain could alert the user: "Your agent has been revoked by [BankName] at [time]." Likewise, if the agent is running some service, it might listen for such events to know it should stop acting (though the contract will anyway block it, but it's good for the agent to know it's been de-authorized).
15. **Post-Revocation Effects:** After this on-chain update:
 16. Any **pending transactions** that the agent might have had in flight (not yet confirmed) will either be rejected or have no effect if they confirm after the revocation, since the contract state now says no authorization. (In some blockchain platforms, ordering is deterministic in a block, so the revocation might be processed before an agent's malicious action in the same block if the miner or validator orders it that way. In others, if a malicious transaction sneaks in the same block just before the revocation, it might execute - the design could consider requiring a *challenge period* or incorporate real-time monitoring to minimize this window. These are implementation nuances that can be addressed with appropriate consensus rules or transaction prioritization for revocations in critical systems.)
 17. The **agent is effectively barred** from further action on behalf of the user. If the agent still tries to act (e.g. tries to transfer assets from user's account contract), the contract's check `isAuthorized(user, agent)` will now return false and the action will be blocked. If the blockchain has native delegate transactions, those will fail validation.
 18. The **user's account is now without an active agent**. In many scenarios, this is essentially a *freeze* of delegated capabilities. If the user themselves still has their own key and it wasn't compromised, they can continue to operate their account normally (the authority did not freeze the user, only removed the agent). If the user had lost their key, they might not be able to act until they recover it or reset credentials via the authority (which could be a separate process, possibly outside the scope of the core project but facilitated by it - e.g., the authority might allow the user to register a new key after some identity verification, which could be another function like `resetUserKey(user, newPubKey)` only callable by that authority under strict conditions).
 19. If the agent was malicious, the damage is limited from the time of revocation onwards. The system may provide logs to investigate what the agent did up to that point. Notably, unlike some custodial systems, the authority here could not roll-back or undo actions already taken by the agent (unless combined with something like DTCC's clawback system [6] in a financial context). The primary goal is to prevent further harm, not retroactively reverse history (though in certain domains like finance, integration with reversible transactions could be considered).
 20. **User Notification and Reassignment:** Typically, the user would be notified out-of-band as well (the bank emails or calls them confirming "we revoked your agent as requested"). The user can later assign a new agent if desired by using their key (assuming they regained control or use a new key). The authority has no say in the choice of the new agent - the authority's role was strictly **protective**, not managerial.

Examples and Use Case Scenarios

Example 1: Banking - Compromised Personal Assistant

Alice uses a blockchain-based bank. She has a hardware wallet that she keeps offline, and she delegates daily transaction authority to a fintech app (the agent) that helps pay her bills. The bank is configured as the revocation authority on the ledger for Alice's account. One day, Alice's phone (running the fintech app) is stolen. She's worried the thief might access the agent's keys stored in the app. Alice contacts the bank's fraud hotline immediately. The bank's officer verifies Alice's identity via security questions (off-chain) and then uses the bank's revocation key to broadcast a **revocation transaction** on the blockchain for Alice's account. Within minutes, the agent app's key is flagged as revoked on-chain. The thief attempts to use the agent app to transfer funds from Alice's account, but the blockchain's smart contract rejects the transaction because the agent is no longer authorized. Alice's funds remain safe. Later, Alice purchases a new phone and could authorize a new agent (or the same app with new keys) when she's ready, using her own master key to sign the new delegation. Throughout this, the bank could **not** take money or assign itself as Alice's agent - it only had the authority to cut off the compromised agent, which it did in a transparent, recorded manner. This example highlights mitigation of key compromise in a financial context.

Example 2: Voting - Proxy Abuse Prevention

Bob is an elderly voter who delegated his voting power to his niece Carol via a blockchain e-voting platform for municipal elections. The Election Commission is the designated revocation authority (per the platform's rules) - their role is normally passive unless a serious issue arises. Bob later learns that Carol has been coerced by someone to vote in a way that goes against Bob's will, and Bob can no longer communicate with Carol freely. Bob contacts the Election Commission and reports potential proxy coercion. The Commission reviews evidence and, following the legal process, decides to revoke Carol's proxy voting authority for Bob. They issue a revocation on-chain using their Commission key. Carol's ability to vote on Bob's behalf is immediately rescinded; any vote cast by Carol for Bob after that point will not count. Bob is then either given an emergency way to vote directly (perhaps using a one-time code or at a polling station) or his vote is simply not cast this time (depending on policy). This demonstrates how an oversight body can intervene to protect the integrity of a vote while still using a blockchain to record delegation. The Commission did not assign any new proxy; Bob could later assign someone else for future elections if desired. All actions are logged on-chain, providing transparency that the revocation occurred according to protocol.

Example 3: Defense - IoT Drone Delegation

Colonel Diaz controls a fleet of defense drones. She has a private key that can issue commands to them via a blockchain-based command system. To react faster, she delegates command authority for routine reconnaissance missions to an AI agent (with its own key) by recording this on the military's private blockchain: AgentAI is authorized under Col. Diaz for the "Recon Mission" command category. The military's central command (or a secure autonomous watchdog system) is set as the revocation authority. One day, the AI agent starts behaving erratically (perhaps it's compromised or has a glitch). It begins redirecting drones outside of approved zones. Central Command detects this. Using their revocation key, they revoke AgentAI's authority for Col. Diaz on the ledger. Instantly, AgentAI's further commands are refused by drone control smart contracts (they check the ledger before execution). The errant drones either revert to manual control or stop. Catastrophe is averted. The important aspect: Central Command could not take direct control of Col. Diaz's drones (unless that was separately authorized); they only removed the AI agent's power. Col. Diaz or her superiors would then investigate and possibly issue new delegation or operate manually. This scenario shows the system's utility in a high-speed operational setting, adding a security override for autonomous agents.

Example 4: Civic - Digital Power of Attorney

Dana grants a digital power-of-attorney to her lawyer Erin to manage her estate on a government-run blockchain registry (covering property title transfers, etc.). The government's judicial branch is the assigned revocation authority for such delegations (to supervise and prevent abuse). This delegation is recorded on chain: Erin can act as Dana in property-related smart contracts. Later, a dispute arises and a court finds that Erin is abusing Dana's trust. The judge orders revocation of Erin's power-of-attorney. The judicial branch uses its key to revoke the delegation on-chain. Erin's access is immediately suspended - she cannot transfer any of Dana's property tokens. Dana (or a court-appointed new guardian) can later assign a different agent as needed. Here, the blockchain ledger, combined with a legal authority's limited intervention right, provides a robust framework for digital representation and its termination when necessary.

Example 5: Estate Management - Deceased Principal Scenario

Alice designates her lawyer, Bob, as her agent authorized to manage real-estate assets on a blockchain-based property registry. The probate court is recorded as the revocation guardian. Alice explicitly pre-registers a revocation condition "principal_deceased," including a cryptographic hash of a notarized digital death certificate as a required verification document. After Alice's passing, the probate court receives and verifies her official death certificate off-chain. Using its revocation-only key, the court issues a revocation transaction to the blockchain referencing the hash of the notarized certificate. The blockchain smart contract validates the guardian's signature and matches the provided hash against the pre-recorded on-chain revocation condition. The agent's authorization is immediately revoked, safeguarding Alice's assets. Bob can no longer act on behalf of Alice, and authority transfers to the legally appointed executor or heirs, who may authorize new agents.

Example 6: Military - MIA (Missing in Action) Scenario

Major Chen authorizes AgentAI to autonomously manage certain sensitive communications during overseas deployment. The military command, as revocation guardian, registers an explicit revocation condition "principal_MIA" specifying automatic revocation after 12 months if Chen is declared officially missing. Chen is subsequently declared MIA. After the predefined period passes without confirmation of his status, the military command receives an official MIA report. Military command verifies and hashes the report off-chain and submits the revocation transaction referencing the hash and the condition "principal_MIA." Smart contract logic revokes AgentAI's authorization immediately, ensuring no misuse of Chen's delegated authority during uncertainty.

Example 7: Healthcare - Mental Incapacitation Scenario

Dr. Garcia grants her spouse access to manage her medical records on a blockchain healthcare network as her authorized agent. She explicitly defines the revocation condition "principal_mentally_incapacitated," requiring a court ruling as verification. Dr. Garcia subsequently suffers a traumatic brain injury resulting in a coma. A court issues a ruling declaring her mentally incapacitated. Upon receipt and verification of this ruling, the healthcare authority, designated as revocation guardian, submits a blockchain revocation transaction referencing the court ruling hash. The blockchain system revokes her spouse's access promptly, protecting sensitive medical data and allowing court-appointed guardians to manage subsequent representation.

Example 8: Cybersecurity - Compromised Principal Scenario

Raj authorizes a cybersecurity AI agent to manage network operations on his blockchain-based system. Raj predefines "principal_compromised" as a revocation condition to handle emergencies explicitly, verified via a signed compromise declaration. When Raj detects his key compromise, he immediately signs a cryptographic declaration stating this fact and sends it securely to the cybersecurity authority guardian. The guardian verifies this declaration, issues a revocation transaction referencing the declaration's hash, and the blockchain ledger immediately revokes the AI agent's authorization. Raj's compromised key can no longer be leveraged maliciously through the agent, significantly reducing potential damage.

Example 9: Personal Security - Kidnapped Principal Scenario

Elena delegates her financial management authority to a blockchain-based AI agent. Given security concerns, Elena explicitly defines "principal_kidnapped" as a revocation condition, requiring verification through an official police report. When Elena is kidnapped, her family contacts law enforcement authorities, who issue an official kidnapping report. Elena's designated revocation guardian (her primary bank) verifies the kidnapping report, hashes it, and issues the revocation transaction on the blockchain referencing this verification. Elena's AI agent's authorization is promptly revoked, preventing kidnappers from coercing or exploiting the agent's access to her finances.

Example 10: Corporate - Unreachable Principal Scenario

Martin, an executive, designates an assistant as an authorized agent to act on his behalf for corporate contracts managed via blockchain. Martin defines an explicit revocation condition "principal_unreachable" with a predefined duration of 6 months, requiring official verification of unreachability. After Martin unexpectedly becomes unreachable during overseas travel, the duration of 6 months passes without any verified communication. The company's legal department, acting as the revocation guardian, verifies Martin's status via official reports, hashes the verification, and submits a revocation transaction. The smart contract immediately revokes Martin's agent, ensuring corporate decisions and contracts remain secure and controlled.

Implementation Variations

- **Multiple Agents and Partial Revocations:** In some implementations, a user might have multiple agents concurrently, each for different sub-tasks or with different permissions. The ledger could store a list of agents or a set of (agent, permission) pairs. The revocation authority's key might be able to revoke *one specific agent* at a time (which is likely), or possibly all agent authorizations for a user in one go (e.g., a function `revokeAllAgents(user)` that the authority can call in case of widespread compromise). The design can accommodate granular revocation. For example, if a user had both a finance agent and a voting proxy, the financial regulator could revoke the finance agent without affecting the voting proxy (which is overseen by a different authority), thanks to domain separation.
- **Hierarchical or Social Recovery as Complementary:** This project focuses on a *central authority* as revocation party, but it could be extended or complemented by social/hierarchical recovery systems. For instance, instead of a single authority key, the revocation permission could be given to a **multi signature group** that includes, say, a few trusted community members or a combination of the institution and some of the user's guardians. This would ensure that the power is not vested in a single entity to avoid abuse. The core idea of limiting to revocation-only still applies, even if multiple parties hold pieces of that power.
- **Integration with Identity Systems:** Each agent or user could be linked to off-chain identity information. The ledger might store merely keys, but the institution likely knows who the user and agent are in the real world (especially in regulated domains). This mapping between blockchain addresses and real identities could be managed by the authority but need not be public on-chain. The patent, however, is agnostic to whether identities are pseudonymous or known - it deals with keys and permissions primarily.
- **Temporal Revocation and Restoration:** The contract could allow for temporary suspension of an agent as well. For example, an authority might suspend an agent pending investigation, and if the agent is cleared, there could be a mechanism (possibly requiring the user's consent as well) to restore the agent without needing a fresh assignment. This is a policy-layer detail that could be implemented with an `active` flag vs. `revoked` status difference. But one must be careful - if the user's key is compromised, one wouldn't want an attacker to be able to quickly reassign a malicious agent. Likely, any restoration would involve the user's participation.
- **No Authority Use Case:** It's worth noting the system still functions even if the authority never intervenes. In normal times, the overhead is minimal - just an extra check on transactions to see if an agent is authorized. The user could revoke their agent directly if they have control. The authority's role is dormant until needed. This is good from a decentralization standpoint: it doesn't actively manage daily operations, only emergencies.

- **Domain-Specific Logic:** The “domain” concept can be implemented in various ways. It could be a simple tag in the registry, or separate registry contracts for different domains (with potentially different authority keys). For example, one could deploy `BankAgentRegistry` contract (authority = Bank), and a separate `VoteProxyRegistry` (authority = ElectionBoard). In that case, a user who wants both a bank agent and a vote proxy deals with two contracts. This separation can provide clarity and prevent any overlap or confusion of authority. It also means revocation keys are siloed to their domain.
- **Permission Scoping:** The project can include fine-grained permissioning. Instead of an agent being a binary “on/off” for a user, the ledger could encode what exactly the agent is allowed to do. This can be accomplished by linking the delegation to roles or smart contract function selectors. For instance, an agent might only be allowed to call `transfer()` up to a limit, but not `approve()` or other functions, etc. The revocation, in such a case, might target the entire agent or possibly just certain permissions (e.g., downgrade the agent’s permissions). However, to keep the core concept clear, the broad notion is that the authority can remove the agent’s **entire authorization**. Fine grained adjustments could be left to either the user (who could reassign with lesser permissions) or not done via the authority to avoid complexity.

In conclusion, the detailed embodiments show how the system can be constructed and operated to provide a **secure, flexible, and accountable way to manage delegated authority on blockchain** with a crucial failsafe: a trusted party can only **pull the plug** on an agent, but never insert themselves or others in the chain of command. This dramatically reduces the risk of unrecoverable loss or damage in scenarios of key compromise, bridging a critical gap between decentralized user control and practical safety requirements in real-world deployments.

Integration with LLM Normalized Content Schema and Advanced AI Agent Permissions

In further embodiments, the system integrates a structured content schema and novel permission controls to manage AI agents (particularly LLM-driven assistants) with greater precision. Modern AI agents can be unpredictable if given broad authority^[10], so a robust framework is introduced to clearly delineate what an agent **may** do, under what conditions, and how those permissions can be constrained or revoked. This extension borrows principles from the **LLM Normalized Content Schema (LLM-NCS)** - a format emphasizing explicit metadata, versioning, and clarity for AI-consumable content - ensuring that an AI agent’s authorization scope is **unambiguous, transparent, and interpretable** by all parties (including the agent itself).

AI Interaction Attribute (aai)

At the user interface level, we introduce an **aai (Allow AI) attribute** for HTML elements (e.g. form tags, input fields, submit buttons) to signal where autonomous agents are permitted to act. By default, `aai="allow"` is assumed (meaning an AI agent acting on the user’s behalf is authorized to fill or submit the element), but developers or content creators can explicitly set `aai="deny"` on sensitive fields or actions to prevent AI-driven interaction. For example, a payment form might mark the cardholder signature field with `aai="deny"` to require explicit user input, whereas routine fields (address, amount within limits) remain `allow`. This attribute serves as an in-content permission hint that the agent’s UI middleware can parse, complementing the on-chain permissions. In effect, it creates a **dual gate**: the on-chain ledger governs *whether* the agent is authorized in principle, and the aai attributes on a webpage or app UI govern *where* and *how* the agent can execute those powers in practice. Such an approach aligns with providing clear, structured signals to AI, much like LLM-CS provides content metadata that AI can easily interpret. By reading `aai` flags, an agent knows which actions are sanctioned, preventing accidental misuse of its authority in user interfaces.

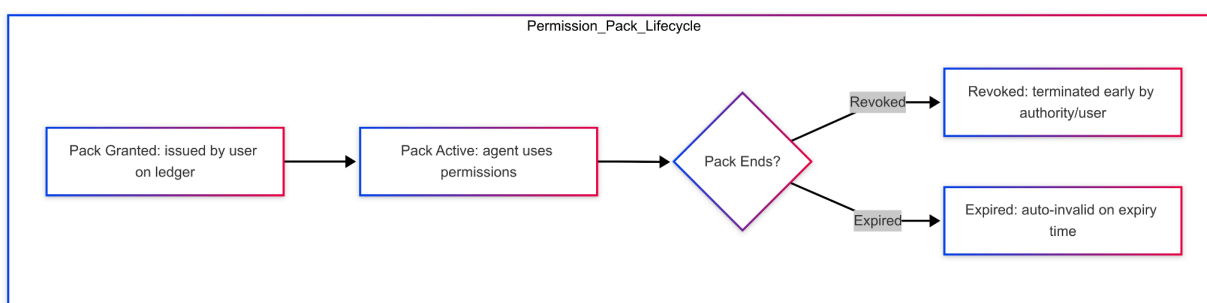
Structured Permission Packs

To granularly define an agent's capabilities, the system uses **Permission Packs**, which are digitally signed, structured certificates of authority recorded on the ledger. A Permission Pack is essentially a bundle of scoped permissions with associated metadata (issuer, issue time, expiration, etc.), akin to a **well-defined object with explicit attributes** describing what the agent can do. Rather than granting an agent unchecked access, the user assembles specific packs that cover only certain data or actions. Two principal categories of Permission Packs are introduced:

- **Sensitive Data Access Packs:** These packs grant the agent **read or write access to private user data** in a controlled manner. For instance, a "Credit Card Pack" might allow an AI shopping assistant to use the user's credit card details for a one-time transaction on an authorized site, or a "Medical Records Pack" could permit a health AI to fetch the user's allergy history from a records database. Each Data Access Pack explicitly enumerates the data scope (e.g. *card ending 1234*, *billing address X*, or *medical record entries from 2015-2020 for doctor Y*), and includes conditions such as **expiration** (after which the pack is invalid) and **non-retention clauses**. A non-retention clause requires the agent to purge the sensitive data from its memory/storage after use or upon pack expiry - ensuring the AI does not indefinitely retain private information. For example, the Credit Card Pack might expire in 10 minutes or after one use, prompting the agent to delete the card number from its working memory once the purchase is complete. These packs are stored as tamper-evident records on the ledger (often as an attachment or hash reference to the actual encrypted data or token), so that anyone (or any process) can verify *if and when* an agent was granted access to a particular data item and whether that access is still valid. The ledger entry for a Data Access Pack would typically include: a unique pack identifier, the agent and user IDs, the data scope description, issuance timestamp, expiration timestamp, and a status flag (active/revoked/expired). By structuring these fields in a standardized way (inspired by LLM-NCS's metadata clarity), an AI agent can programmatically check, "Do I have a valid pack to read X data, and is it fresh and authorized?" at runtime - and get a clear yes/no answer based on the ledger state[10]. If the pack is revoked or expired, the ledger state (and the pack's metadata) makes that immediately apparent, and the agent must refrain from accessing the data. This approach mirrors best practices in AI authorization which recommend treating the agent as a distinct principal with limited credentials, rather than letting it inherit all user privileges[10].
- **Autonomous Action Packs:** These packs authorize the agent to take **preemptive or reactive actions on the user's behalf under specific conditions**, effectively encoding a contract for agent "paternalism" or guardian-like behavior. Each Action Pack defines a scenario or trigger and the permitted response. For example, a *Fraud Prevention Pack* might say: "*If the agent detects a likely fraud (e.g. a transfer above \$X to an untrusted new payee) and the user is unavailable to confirm, the agent may freeze the account or delay the transaction.*" Similarly, a *Health Emergency Pack* could allow a wearable AI assistant to automatically call emergency services and share the user's GPS location if it detects a critical health anomaly (like heart stoppage or a dangerous fall) while the user is incapacitated. Another example is a *Cybersecurity Pack*: if an AI system monitoring the user's devices notices a cyber-intrusion (ransomware or malware attack), it is authorized to quarantine the device or cut off network access immediately to contain the threat. These Action Packs contain structured triggers (the condition under which action is allowed), the specific action(s) the agent is allowed to execute, and any limits or notifications required. They often include an **expiration or review interval** as well - for instance, a Fraud Prevention Pack might automatically expire after 1 year and require renewal, ensuring that the user and oversight authority periodically re-evaluate the standing orders. Crucially, even when an Action Pack is in force, the agent's autonomy is **bounded** to the scenarios described: the agent cannot arbitrarily take other actions outside that scope. All Action Packs are also recorded on the public ledger as part of the agent's authorization record, allowing real-time audit by the user or third parties. If an agent takes an emergency action, it can

log a reference to the authorizing Pack (by ID) on-chain or in an audit trail, proving that it acted according to a pre-approved permission. This provides accountability and traceability for autonomous agent behavior. By confining proactive powers to what's explicitly documented in a Permission Pack, the system prevents the slippery slope of agents overstepping their mandate - a problem already highlighted in AI safety literature (agents should **never be given broad, unsupervised authority without strict scopes**[10]). The structured nature of the pack (with clear fields for trigger and action) also makes it straightforward for an AI agent to interpret its own allowed behaviors, or for a monitoring service to enforce that the agent *only* does what the pack allows.

The lifecycle of a Permission Pack is illustrated in the diagram below. It begins with a user granting (issuing) the pack, after which the agent may utilize it within its defined scope. The pack can be *revoked* by the user or trusted authority at any time, or *automatically expire* upon reaching its time limit or conditions, at which point the agent's rights under that pack terminate:



When a Permission Pack is revoked or expires, the ledger state is updated to mark it invalid, and the agent (as well as any external service verifying the agent's credentials) will treat it as such. For instance, if an agent tries to use a Data Access Pack after it's revoked, the attempt will fail a ledger check (similar to how a revoked cryptographic certificate is rejected). The design ensures that **delegated permissions are as ephemeral or durable as intended** - they can be tightly time-bound or condition-bound to minimize risk. Importantly, these packs and their statuses live on a **public, tamper-evident distributed ledger**, which means the audit trail of who granted what, and when revocation occurred, is transparent and cannot be secretly altered. This public anchoring is analogous to how the base agent-authorizations were handled in our core design (ledger as source of truth), but now extended to fine-grained permissions.

Agent Autonomy Levels and Paternalism Settings

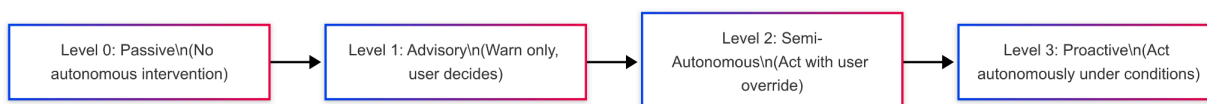
In tandem with Permission Packs, the system allows the **degree of agent autonomy or "paternalism"** to be explicitly specified and recorded as a user preference or policy. Not all users will want an AI agent making decisions on their behalf unilaterally - some may prefer the agent to be passive unless explicitly instructed, while others may welcome the agent intervening to protect them from mistakes or threats. We introduce a graded scale of agent authority that can be set per agent (or per pack), and lodged on the ledger or associated policy files, so that it is **common knowledge** how proactive the agent is allowed to be. For example, the user (or an application default) might choose from levels such as:

- **Level 0 - Passive:** The agent has **no independent initiative**. It only acts when directly commanded by the user and never alters or blocks user-initiated actions. This is a non-paternalistic setting ensuring maximum user autonomy; the agent is essentially an obedient tool with no guardrail behavior. (E.g., the agent would never stop a transaction or send a warning on its own; it waits for user instructions.)
- **Level 1 - Advisory:** The agent can **observe and warn**, but not act unilaterally. It will intervene only in the form of suggestions or alerts. For instance, if the user is about to send funds to an unrecognized

address, the agent might flash a warning or ask “Are you sure? This looks suspicious,” but *will not prevent* the action if the user confirms. This level introduces soft paternalism - the agent acts as a sentry or advisor (helping avoid scams or mistakes) but the final decision remains with the user every time.

- **Level 2 - Semi-Autonomous:** The agent is allowed to **take certain preventative actions on behalf of the user, but with user notification or quick post-action override**. Here the agent has moderate paternalistic authority in predefined situations. For example, the agent might temporarily halt a suspicious bank transfer or put an incoming strange email in quarantine *immediately*, but then promptly notify the user (“I paused this action because it looked like a fraud; tap to approve if it was actually intended”). The user can then either approve the action (in which case the agent executes it or releases the hold) or let it remain blocked. This level seeks to strike a balance: the agent can act fast to mitigate harm, but the user is kept in the loop and can reverse the agent’s decision in short order. Any such agent-initiated intervention would be logged (potentially on-chain or in an audit log) for transparency.
- **Level 3 - Fully Proactive:** The agent has **broad license to act on its own** in the user’s interest under the constraints of its Permission Packs. This is a high paternalism setting, where the user effectively delegates trust to the agent to handle critical issues autonomously. In this mode, if the conditions in an Action Pack are met, the agent doesn’t need additional confirmation to execute the response (e.g., it will freeze the account at the first sign of a qualifying fraud without asking). The agent might even preemptively refuse commands from the user that appear catastrophically harmful (for instance, if an attacker coerces the user into transferring all funds, a fully-proactive guardian agent might block it and alert authorities - a scenario that borders on **safety override**). Such strong autonomy is likely to be used only in domains where the cost of not acting immediately is irreversible loss or harm, and where the user explicitly opts into this protection knowing it constrains their freedom. The parameters of what an agent can override would be clearly delineated in its Action Packs and in policy (e.g., “*Agent may countermand user’s transaction if it matches known scam patterns and user is incapacitated or under duress*”).

These levels create a **paternalism gradient** from purely user-driven to highly agent-driven. The chosen level (or a numeric code representing it) can be recorded on-chain alongside the agent’s other credentials or in a parallel policy registry, ensuring that the agent’s behavior profile is verifiable. By anchoring this setting on the ledger, a regulator or oversight entity could later verify, for example, that a given agent was *supposed* to be in advisory-only mode when it took a certain action, flagging a violation if it overstepped. It also means users can publicly prove the intended autonomy level of their agents (which could be relevant for liability or compliance - e.g., a fintech service might require that all customer agents are at most Level 1 to prevent unsanctioned trades). The following diagram illustrates conceptually the spectrum of agent authority:



Notably, **Permission Packs and paternalism levels work in tandem**: the packs define *what* the agent can access or do, and the autonomy level defines *how* the agent may execute those permissions (silently, with approval, or automatically). A fully proactive agent might be granted a Fraud Prevention Pack to actually freeze accounts, whereas a Level 1 advisory agent could have a similar pack but only with the power to *recommend* freezing (leaving the final action to the user or a higher-level process). By combining these mechanisms, the system achieves a nuanced permission handling model: it’s not just binary “agent or no agent,” but a configurable spectrum of trust and authority, all of which is enforceably documented.

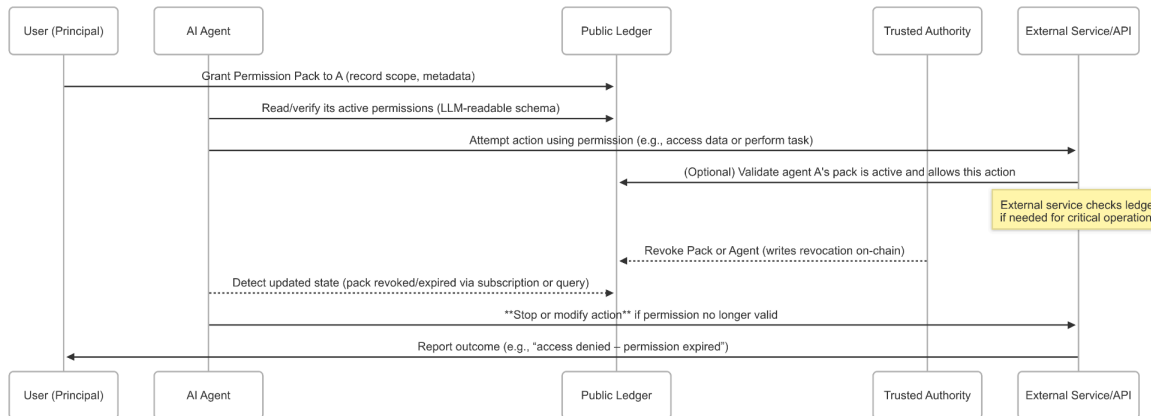
Public Ledger as the Source of Truth for Agent Scopes

All the above enhancements are built on the foundation of a public ledger acting as the **source of truth** for agent permissions and their status. The ledger entries for an agent now encompass: the agent's identity linked to the user, a list of active Permission Packs (each with its detailed scope and metadata), and the agent's autonomy level setting. This constitutes the agent's *authorization schema* on-chain. Because it's on a blockchain or similar distributed ledger, these records are tamper-evident and globally verifiable. The inherent transparency means that if, for example, an agent's Data Access Pack expires at time T, anyone inspecting the ledger after time T will see that it's no longer valid (and the agent should accordingly be unable to use it). Likewise, a revocation authority (the trusted third party in our core design) can invoke a **pack revocation transaction** on the ledger to immediately strip an agent of a particular Permission Pack (or all packs) without waiting for expiration. The authority's on-chain permissions remain **revocation-only** - it can mark a pack or an entire agent authorization as revoked (similar to the base design where the authority could only remove an agent, not add one). Here, that role extends to *specific scopes*: e.g., a bank acting as revocation guardian might not remove Alice's agent entirely, but could revoke the agent's "wire transfer" Pack if a fraud is suspected, leaving other less sensitive packs intact. Every such change is logged on-chain, creating an audit trail of exactly which powers were revoked, by whom, and when. This granularity enhances security: even if an agent is generally trusted, a targeted revocation of just the dangerous permission can mitigate harm while allowing the agent to continue benign functions.

Another benefit of anchoring Permission Packs on a public ledger is **multi-party interoperability**. Different services or websites can query the ledger to determine an agent's rights without needing direct integration. For instance, if the user's agent tries to access a health record from Hospital B's API, Hospital B's systems (or the agent's request itself) can reference the ledger entry of the Medical Records Pack to confirm the agent indeed has current permission for that data. In essence, the ledger serves as a decentralized **authorization oracle**. It bridges trust across domains: an AI agent roaming across financial services, healthcare providers, and personal devices carries a portfolio of on-chain credentials that each domain can verify. This avoids brittle hard-coding of permissions in each service - instead, everyone trusts the ledger. The agent as a digital entity becomes somewhat like an actor with a **wallet of permission tokens** (the packs), which it can present as needed. When those tokens expire or are revoked, they are no longer valid, period.

The model also aids in agent interpretability and compliance. Since each Permission Pack is structured with metadata (issuer, scope, timestamps, etc.), compliance auditors or automated tools can analyze them easily, just as LLM-NCS structured entries allow automated processing and verification of content updates. The stability of the schema (fields that persist and only evolve in backward-compatible ways) means that as new types of Permission Packs are introduced over time, older agents or services won't break when encountering them - unknown fields can be ignored or new versions of packs can carry version tags, much like how content schema versions signal changes without invalidating old data. We ensure that updating the permission schema "never breaks user-space" expectations in the spirit of robust interface design.

Finally, Figure 7 below outlines the overall integration of these concepts in a unified model. It shows how a user issues Permission Packs to an agent, how the agent and external services consult the ledger for the agent's authorized scope, and how a trusted authority can revoke permissions. The public ledger anchors all the agent's scopes and actions, making the flows transparent and verifiable:



In this sequence, all crucial events (granting, usage, revocation) interface with the ledger as the authoritative source. The AI agent is effectively sandboxed by what the ledger says it can do. By designing the permission records with **clear metadata and schema** (drawing on LLM-NCS principles for naming and structure clarity), we ensure the agent (and any other LLM or AI inspecting the record) can **unambiguously interpret its authorization** - for example, fields like `allowed_actions: ["READ_MEDICAL_DATA"]`, `expires: 2025-12-31T23:59:59Z`, or `paternalism_level: 2` are self-explanatory to an AI parsing the JSON/YAML. The agent doesn't have to infer meaning from unstructured text; it's all declarative. This metadata clarity reduces the risk of miscommunication or error in agent behavior, thereby increasing reliability.

In summary, the integration of LLM-NCS-style schema with the distributed ledger delegation system brings **metadata-rich, fine-grained control** to AI agent authorizations. It extends the original delegation/revocation architecture into a comprehensive framework where not only is *who* can act for the user managed on-chain, but also *what exactly* they can do, *when*, and *how autonomously* they may do it. All these facets are recorded on a tamper-evident ledger for accountability. This model enhances user trust in AI agents by giving users and institutions the tools to precisely tailor and later revoke an agent's powers. It provides a path to safely deploy advanced AI assistants (LLM-based or otherwise) in sensitive domains - the agents can be granted narrow, auditable slices of authority rather than the keys to the kingdom. By coupling strong cryptographic permission controls with schema-driven transparency (inspired by LLM-NCS's emphasis on clarity and stability), we achieve an agent governance system that is both **empowering and safe**: empowering, because users can harness AI assistance in complex tasks via well-defined delegations; safe, because every delegation is bounded, visible, and revocable, with the agent's behavior bound by enforceable rules rather than open-ended trust. All of this advances the state of the art in agent delegation and revocation, moving beyond simple binary authority into a nuanced, next-generation permission management architecture for AI age.

Sources:

- ArcBlock Inc., *"Blockchain delegation"*, US Patent 11,838,405 (issued Dec. 5, 2023) - describes on-chain delegation and user-initiated revocation. [1] [2]
- EP Patent 4018614B1, *"DID delegation/revocation to another DID"* (granted 2024) - illustrates decentralized ID delegation and user revocation of a device via ledger update. [3]
- DTCC (Digital Asset Transfer), US Patent Application 20250078065A1 (published 2025) - proposes hierarchical key management with a **transfer agent** role for clawback of transactions when a delegate is compromised. [5] [6]
- LedgerDomain Inc., *"Credential revocation leveraging private keys on keystores..."*, US Patent 12,271,464 (issued Apr. 8, 2025) - shows enterprise blockchain where an admin can revoke a user's credential via ledger entry. [8] [9]
- Additional background on blockchain revocation and delegation mechanisms is referenced in the above description for contextual support and clarity. [4] [8]

[1] [2] U.S. Patent for Blockchain delegation Patent (Patent # 11,838,405 issued December 5, 2023) - Justia Patents Search
<https://patents.justia.com/patent/11838405>

[3] EP4018614B1 - Did delegation/revocation to another did - Google Patents
<https://patents.google.com/patent/EP4018614B1>

[4] [5] [6] [7] US20250078065A1 - Method, apparatus, and computer-readable medium for confederated rights and hierarchical key management - Google Patents <https://patents.google.com/patent/US20250078065A1/en>

[8] [9] U.S. Patent for Credential revocation leveraging private keys on keystores read by provisioned devices Patent (Patent # 12,271,464 issued April 8, 2025) - Justia Patents Search
<https://patents.justia.com/patent/12271464>

[10] "Handling AI Agent Permissions," Stytych Blog, 22 Jan 2025.
<https://stytych.com/blog/handling-ai-agent-permissions/>